

Physics-Informed Neural Networks for Incompressible Laminar Flow Around a Cylinder

Chi-Han Chiu, Chen-Yan Juang, Ming-Yang Wu, and Jing-Hua Chang
 Department of Electrical and Computer Engineering
 University of California San Diego
 {c8chiu, chjuang, miw090, jic184}@ucsd.edu

USER: Abstract—

write
ab-
stract

150
words,

cover-
ing;
prob-
lem,
PINN

for-
mu-
lation,

Re=10
re-
sult,

com-
par-
i-
son
to
Rao
base-
line

I. Introduction

II. Problem Formulation

III. Methodology

A. Mixed-Form PINN Formulation

The 2D incompressible Navier–Stokes equations in stress form are cast following [4]. A stream function ψ is introduced so that

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x}, \quad (1)$$

which satisfies the continuity equation $\partial_x u + \partial_y v = 0$ identically and eliminates one residual term.

Auxiliary stress variables σ_{11} , σ_{22} , σ_{12} are introduced to reduce the PDE order from second to first, giving the constitutive relations

$$\sigma_{11} = 2\mu \partial_x u, \quad \sigma_{22} = 2\mu \partial_y v, \quad \sigma_{12} = \mu(\partial_y u + \partial_x v). \quad (2)$$

The momentum residuals are

$$f_u = \rho(u \partial_x u + v \partial_y u + \partial_t u) + \partial_x p - \partial_x \sigma_{11} - \partial_y \sigma_{12}, \quad (3)$$

$$f_v = \rho(u \partial_x v + v \partial_y v + \partial_t v) + \partial_y p - \partial_x \sigma_{12} - \partial_y \sigma_{22}. \quad (4)$$

The constitutive residuals enforce Eqs. (2) as soft constraints in the loss.

B. Network Architecture

The physics-informed neural network (PINN) is a fully connected multilayer perceptron (MLP) $\mathcal{N}_\theta : \mathbb{R}^3 \rightarrow \mathbb{R}^5$ that maps the space–time coordinate (x, y, t) to the five output fields $(\psi, p, \sigma_{11}, \sigma_{22}, \sigma_{12})$. Velocity components are recovered via automatic differentiation of ψ using Eq. (1).

The network comprises 7 hidden layers each with 50 units and hyperbolic tangent (tanh) activations, for a total of 15 755 trainable parameters. Weights are initialised with truncated Xavier (Glorot) uniform initialisation using standard deviation $\sqrt{2/(n_{\text{in}} + n_{\text{out}})}$, truncated at 2σ , matching the TensorFlow default used in the original implementation [4].

C. Boundary and Initial Conditions

The spatial domain is $x \in [0, 1.1]$ m, $y \in [0, 0.41]$ m with a cylinder of radius $r = 0.05$ m centred at $(0.2, 0.2)$ m. The fluid parameters are $\rho = 1.0$ kg/m³, $\mu = 0.005$ Pa·s, giving $Re = 10$.

The inlet profile is a pulsatile parabolic flow,

$$u(0, y, t) = \frac{4U_{\text{max}} y (H - y)}{H^2} \left(\sin\left(\frac{2\pi t}{T} + \frac{3\pi}{2}\right) + 1 \right), \quad (5)$$

with $U_{\text{max}} = 0.5$ m/s, $H = 0.41$ m, and $T = 1.0$ s. No-slip conditions $u = v = 0$ are imposed on the top, bottom, and cylinder walls. A zero-stress (natural) outlet condition is applied at $x = 1.1$ m. Initial conditions set $u = v = p = 0$ at $t = 0$.

Each boundary condition and the initial condition are enforced as mean squared error terms in the composite loss,

$$\mathcal{L} = \mathcal{L}_f + \beta_{\text{wall}} \mathcal{L}_{\text{wall}} + \beta_{\text{inlet}} \mathcal{L}_{\text{inlet}} + \mathcal{L}_{\text{outlet}} + \mathcal{L}_{\text{ic}}, \quad (6)$$

where $\beta_{\text{wall}} = \beta_{\text{inlet}} = 5$ and all other weights are unity.

D. Collocation Point Sampling

Collocation points are drawn using Latin hypercube sampling (LHS) [4]. A base set of 80 000 points is drawn uniformly over the space–time domain, supplemented by 21 000 near-wall and wake-refinement points concentrated within two cylinder radii and in the downstream wake region. The total collocation set is approximately 101 000 points.

E. Two-Phase Training

Training proceeds in two phases. In the first phase, the Adam optimiser runs for 5 000 iterations with a fixed learning rate of 5×10^{-4} . In the second phase, the L-BFGS-B algorithm (via SciPy) is applied with up to 100 000 function evaluations, using history size $m = 50$, maximum line-search steps of 50, $\text{ftol} \approx \varepsilon_{\text{mach}}$, and $\text{gtol} = 0$. This two-phase schedule mirrors the TensorFlow training protocol of the original paper [4]; the SciPy L-BFGS-B wrapper was selected over the PyTorch built-in L-BFGS to faithfully reproduce the convergence criterion. Training converged early at approximately 68 000 function evaluations.

USER: The physics-informed neural network (PINN) is a fully

write
in-
tro-
duc-
tion

—
mo-
ti-
vate
PINNs
for
fluid
sim-
u-
la-
tion,

cite
[1],
[2],

TABLE I

Per-snapshot L_2 relative error (%) for u , v , and spatially demeaned p , $Re = 10$. reproduce is the SciPy L-BFGS-B baseline; GPU-resident is the PyTorch L-BFGS run. Selected snapshots shown; best per column in bold.

t (s)	reproduce			GPU-resident		
	ε_u	ε_v	ε_p	ε_u	ε_v	ε_p
0.10	10.19	41.74	3.44	11.87	47.50	2.42
0.20	5.63	24.39	4.29	6.77	27.27	4.23
0.30	4.25	16.47	9.06	4.93	18.61	9.04
0.40	4.44	13.29	4.45	4.50	14.70	6.33
0.50	4.81	12.60	70.08	4.72	13.57	63.86
Mean	5.30	19.86	10.79	5.94	22.23	10.31

IV. Results

A. Reference Solution

The reference solution is generated by a Chorin fractional-step projection method [5] on a uniform 220×82 grid with time step $\Delta t = 0.001$ s over the interval $t \in [0, 0.5]$ s, yielding 51 snapshots at $\Delta t_{\text{snap}} = 0.01$ s. The physical parameters and boundary conditions match those described in Section III exactly.

B. Quantitative Error Comparison

The L_2 relative error is defined as

$$\varepsilon = \frac{\|\mathbf{u}_{\text{pred}} - \mathbf{u}_{\text{ref}}\|_F}{\|\mathbf{u}_{\text{ref}}\|_F}, \quad (7)$$

where $\|\cdot\|_F$ denotes the Frobenius norm over all grid points at a given snapshot. Errors are evaluated over the 41 snapshots with $t \geq 0.1$ s to exclude the initial transient during which the solution magnitude is near zero.

The original Rao et al. [4] implementation cannot be faithfully reproduced post-hoc—only a trained TensorFlow checkpoint is available, and its training conditions cannot be exactly recovered. We therefore use our strict re-implementation of the Rao et al. protocol with SciPy L-BFGS-B, denoted reproduce, as the baseline, and compare it against the GPU-resident PyTorch variant whose L-BFGS loop runs entirely on the GPU via torch.optim.LBFGS. Both are evaluated against the same Chorin CFD solution; Table I reports only relative errors, not reference-field values.

C. Discussion

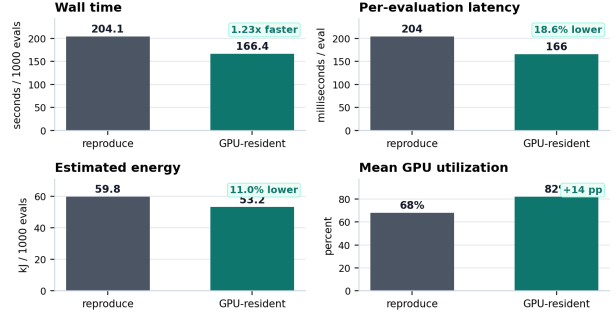
The GPU-resident variant attains accuracy close to the reproduce baseline: the mean-error gap is 0.64 percentage points in ε_u (5.94% vs. 5.30%) and 2.37 points in ε_v (22.23% vs. 19.86%). This gap is attributed to the different L-BFGS backends—SciPy L-BFGS-B for reproduce versus PyTorch’s native GPU LBFGS for GPU-resident, which differ in line search and stopping behaviour—together with minor random-seed and collocation-sampling differences. The GPU-resident loop keeps all optimizer state on the

TABLE II

Backend benchmark: 1000 L-BFGS function evaluations from a common Adam checkpoint on one RTX 3090, $Re = 10$. Only the L-BFGS backend differs. Energy is estimated as mean GPU draw multiplied by wall-clock time.

Metric	reproduce	GPU-resident	Change
Wall time	204.1 s	166.4 s	1.23× faster
Eval latency	204 ms	166 ms	18.6% lower
Energy	59.8 kJ	53.2 kJ	11.0% lower
GPU utilization	68%	82%	+14 pp

Controlled L-BFGS backend benchmark on RTX 3090



Both runs start from the same Adam checkpoint and execute 1000 L-BFGS function evaluations. Energy is mean GPU draw times wall-clock time.

Fig. 1. Controlled backend comparison for 1000 L-BFGS evaluations. The GPU-resident backend reduces wall-clock time, per-evaluation latency, and estimated energy while increasing mean GPU utilization.

GPU and avoids the per-evaluation host-device transfer that the SciPy wrapper incurs on every function evaluation.

To isolate this effect we ran a controlled benchmark: starting from a common Adam checkpoint, 1000 L-BFGS function evaluations on the same RTX 3090, varying only the backend (SciPy L-BFGS-B versus PyTorch’s native GPU LBFGS). Table II and Fig. 1 summarize the result. The GPU-resident backend completes the 1000 evaluations in 166 s versus 204 s for SciPy (1.23× faster), reduces per-evaluation latency by 18.6%, and increases mean GPU utilization from 68% to 82%. Although its instantaneous GPU draw is higher, the shorter runtime lowers the estimated benchmark energy from 59.8 to 53.2 kJ. The SciPy wrapper runs its L-BFGS-B linear algebra on the host and copies parameters and gradients between device and host on every evaluation, whereas the GPU-resident loop keeps optimizer state on the GPU. The per-evaluation floating-point work is identical, so the measured gap is backend overhead; its magnitude is specific to this network size, collocation count, and hardware.

The early-time snapshots ($t = 0.10$ – 0.20 s) exhibit elevated errors in both methods because the pulsatile inlet imposes the most rapid variation; the PINN must satisfy a stiff constraint between the initial condition ($u = v = 0$ at $t = 0$) and the imposed inlet profile, which is a known source of difficulty for time-domain PINNs [6].

Pressure error is included after spatial demeaning to remove the arbitrary pressure gauge. The mean pressure errors are comparable (10.79% for reproduce and 10.31% for GPU-resident), although the final snapshot has a large pressure error in both runs. We therefore use pressure as a secondary diagnostic rather than as the primary ranking criterion.

D. Causal Training

To test whether temporal-causality weighting improves the transient solution, we replace the PDE-residual term with the causally-weighted form of Wang et al. [6]. The collocation points are binned into $M = 32$ equal-duration temporal slices, and bin i receives a stop-gradient weight

$$w_i = \exp\left(-\frac{\epsilon}{M} \sum_{k < i} L_k\right), \quad (8)$$

where L_k is the mean residual of bin k . The exponent is normalized by M so that ϵ is independent of the bin count, and at $\epsilon = 0$ the loss reduces exactly to the unweighted residual of Eq. (7)’s underlying objective. After the Adam phase the bin weights are frozen as static constants during the L-BFGS phase, keeping the quasi-GPU-Newton objective stationary. We compare this frozen polish against a uniform polish that instead optimises the unweighted true loss (all bins equally weighted) during the L-BFGS phase.

Table III reports the developed-window error ($t \geq 0.1$ s) for causal Adam initialisations at $\epsilon \in \{30, 50\}$, each polished with either a frozen-weight or a uniform-weight L-BFGS phase, alongside the reproduce and GPU-resident baselines.

Two findings stand out. First, at both ϵ values the uniform L-BFGS polish consistently beats the frozen (rain-polish on every field: at $\epsilon = 50$ the velocity errors drop from 5.34%/19.86% (frozen) to 4.96%/19.21% (uniform), and the unweighted true loss is also lower (1.5×10^{-4} versus 2.1×10^{-4}). A causal Adam phase supplies a good cylinder initialisation, but the subsequent L-BFGS reaches a more accurate field when it optimises all temporal bins equally rather than retaining the causal down-weighting of late times. Second, the best configuration—causal $\epsilon = 30$ with a uniform polish ($\epsilon_u = 4.64\%$, $\epsilon_v = 18.60\%$)—surpasses the reproduce baseline (5.30%/19.86%) and approaches the original Rao TF checkpoint (4.75%/18.47%), even though that checkpoint cannot be reproduced.

These results are exploratory, not a controlled ablation. The causal runs use a more aggressive schedule (20,000 Adam iterations at learning rate 10^{-3} with a plateau scheduler) than the baselines, so the gain conflates the causal weighting with the longer, scheduled Adam phase; an iso-budget sweep ($\epsilon \in \{0, 2, 5\}$ under matched settings) is still required to isolate the causal-weighting contribution. We also note that the causal curriculum did not fully propagate within the Adam budget ($\min_i w_i \approx 0.77$ – 0.85), and that $\epsilon = 30$ and $\epsilon = 50$ converged to nearly identical causal loss, so ϵ is weakly determined in this regime.

V. Conclusion

References

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [2] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, “Physics-informed machine learning,” *Nature Reviews Physics*, vol. 3, pp. 422–440, 2021.
- [3] M. Schäfer, S. Turek, F. Durst, E. Krause, and R. Rannacher, “Benchmark computations of laminar flow around a cylinder,” in *Flow Simulation with High-Performance Computers II*, ser. Notes on Numerical Fluid Mechanics (NNFM). Vieweg+Teubner Verlag, 1996, vol. 48, pp. 547–566.
- [4] C. Rao, H. Sun, and Y. Liu, “Physics-informed deep learning for incompressible laminar flows,” *Theoretical and Applied Mechanics Letters*, vol. 10, no. 3, pp. 207–212, 2020, arXiv:2002.10558.
- [5] A. J. Chorin, “Numerical solution of the Navier–Stokes equations,” *Mathematics of Computation*, vol. 22, no. 104, pp. 745–762, 1968.
- [6] S. Wang, S. Sankaran, and P. Perdikaris, “Respecting causality for training physics-informed neural networks,” *Computer Methods in Applied Mechanics and Engineering*, vol. 421, p. 116813, 2024.

USER:
write
con-
clu-
sion
—
sum-
ma-
rize
Re=10
fi-
delity,
gaps
vs
Rao
pickle,
planned
ex-
ten-
sions
(Re=100
ar-
chi-
tec-
ture
ab-
la-
tion)

fig:
field
com-
par-
i-
son
of
u,
v,
p
at
t=0.3
0.4,
0.5
s
for
GPU-
resident
vs
re-
pro-
duce
vs
Chorin
ref-
er-
ence
rain-
bow
col-
ormap
cylin-
der
masked

add
iso-
budget
causal
ab-
la-
tion
($\epsilon =$
0/2/5,
con-

TABLE III
 $Re = 10$ developed-flow error ($t \geq 0.1$ s, 41 snapshots). Mean is the per-snapshot relative L_2 averaged over the window; Global is the Frobenius L_2 over all space-time points; Loss is the final unweighted training loss (PDE residual + BC/IC). Causal runs use $M = 32$ temporal bins; frozen keeps the Adam-end causal weights during L-BFGS, while uniform optimises the unweighted loss. Pressure is spatially demeaned. Best per column in bold.

Run	Loss	Mean L_2 (%)			Global L_2 (%)		
		ε_u	ε_v	ε_p	ε_u	ε_v	ε_p
reproduce (baseline)	2.1×10^{-4}	5.30	19.86	10.79	4.47	14.75	27.92
GPU-resident	2.9×10^{-3}	5.94	22.23	10.31	4.75	16.44	24.42
causal $\epsilon=30$, frozen	2.3×10^{-4}	5.22	20.38	11.03	4.36	15.02	28.49
causal $\epsilon=30$, uniform	1.5×10^{-4}	4.64	18.60	8.89	3.93	13.28	22.51
causal $\epsilon=50$, frozen	2.1×10^{-4}	5.34	19.86	10.10	4.47	15.05	25.83
causal $\epsilon=50$, uniform	1.5×10^{-4}	4.96	19.21	9.70	4.14	13.93	25.30